# Kubernetes Day 3 – Student Lab Guide: YAML, Recreate and Rolling Updates

## Before you start

In this lab you will describe applications in YAML files, update them, and see the difference between two update strategies. Type every command yourself; do not copy-paste blindly. After each step, compare what you see with **You should see**.

**You need:** Docker Desktop with Kubernetes enabled, a terminal (PowerShell, Terminal or Git Bash) and a text editor such as VS Code.

**Readiness check (1 minute)**

```
kubectl config use-context docker-desktop
kubectl get nodes
```

**You should see:** one node called `docker-desktop` with status `Ready`. If not, open Docker Desktop, go to Settings, then Kubernetes, tick **Enable Kubernetes**, and wait for the green icon.

**Make a working folder** and stay inside it for the whole lab. All YAML files you create go here.

```
mkdir k8s-day19
cd k8s-day19
```

**Rules for the lab**

- Use **2 spaces** for indentation in YAML, never the Tab key.
- Ask for help when a Pod is stuck for more than one minute; the command `kubectl describe pod <name>` usually shows why.

## Part 1: Imperative vs declarative

**Idea:** there are two ways to tell Kubernetes what you want. *Imperative* means giving a command ("create this"). *Declarative* means writing the desired result in a file ("I want 3 nginx Pods") and letting Kubernetes make it true. Teams use files because files can be reviewed, shared and repeated.

## Step 1.1: Create a Deployment with a command

```
kubectl create deployment web --image=nginx --replicas=3
kubectl get deploy,pods
```

- `kubectl create deployment web ...`: asks Kubernetes to create a Deployment named `web` running the `nginx` image with 3 Pods.
- `kubectl get deploy,pods`: lists Deployments and Pods together. `deploy` is short for `deployments`.

**You should see:** `web 3/3` under Deployments, and three Pods named like `web-6d4cf56db6-abcde`, all `Running`.

## Step 1.2: Look at what Kubernetes generated

```
kubectl get deployment web -o yaml
```

`-o yaml` prints the whole object as YAML. Find these four top-level keys: `apiVersion`, `kind`, `metadata`, `spec`. You are about to write them by hand. (`status` at the bottom is filled in by Kubernetes; you never write it.)

## Step 1.3: Clean up

```
kubectl delete deployment web
```

This deletes the Deployment, and Kubernetes automatically removes its Pods too.

**Checkpoint question:** if you delete one Pod of a Deployment, what happens? Write your guess, then test it later in Part 2.

**Imperative vs declarative**

| | Imperative (commands) | Declarative (files) |
|---|---|---|
| You provide | A command | A YAML file |
| Can you repeat it exactly? | Hard | Yes, apply the same file |
| Can it be reviewed in Git? | No | Yes |
| Best for | Quick experiments | Real applications |

# Part 2: Writing YAML manifests

## YAML in 1 minute

- Indent with **2 spaces**. Indentation shows which lines belong together.
- `key: value` is a map entry. A line starting with `-` is a list item.
- `#` starts a comment.

Every Kubernetes file has the same four top-level fields:

| Field | Meaning |
|---|---|
| `apiVersion` | Which API handles this object (`apps/v1` for Deployments, `v1` for Services) |
| `kind` | The type of object |
| `metadata` | Its name and labels |
| `spec` | What you want (the desired state) |

## Step 2.1: Write the Deployment

Create a file called `deployment.yaml` in your `k8s-day19` folder:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

```
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              cpu: 200m
              memory: 128Mi
```

**Understand every part**

- `replicas: 3` : how many identical Pods you want.
- `selector.matchLabels` : how the Deployment finds its own Pods. It **must match** the labels in `template.metadata.labels` .
- `template` : the blueprint for each Pod. Everything inside `template.spec` describes the container.
- `image: nginx:1.25` : always include a version tag. Today's update demos need a tag that changes.
- `requests` : the minimum CPU and memory reserved for the Pod ( `50m` = 5% of one CPU core). `limits` : the maximum it may use.

## Step 2.2: Check, then apply

```
kubectl apply -f deployment.yaml --dry-run=client
kubectl apply -f deployment.yaml
kubectl get deploy,rs,pods -l app=web
```

- `--dry-run=client` : checks the file without creating anything.
- `apply -f` : makes the cluster match the file (creates it, or updates only what changed).
- `-l app=web` : shows only objects with the label `app=web` .

**You should see:** one Deployment, one ReplicaSet and three Pods. The chain is Deployment, then ReplicaSet, then Pods.

## Step 2.3: Write the Service

Create `service.yaml` :

```
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

A Service gives the Pods one stable address. It finds Pods by **label** ( `selector: app: web` ), not by IP, so it keeps working when Pods are replaced.

- `port` : the Service's own port inside the cluster.
- `targetPort` : the port the container listens on.
- `nodePort` : the port opened on your machine (30000–32767).

```
kubectl apply -f service.yaml
kubectl get svc web-svc
kubectl get endpoints web-svc
curl http://localhost:30080
```

**You should see:** `80:30080/TCP` for the Service, three `IP:80` addresses in endpoints, and the "Welcome to nginx!" page from `curl` (or open `http://localhost:30080` in your browser). If `localhost:30080` does not respond on your machine, run `kubectl port-forward svc/web-svc 30080:80` in a second terminal and try again.

**Checkpoint:** if `kubectl get endpoints web-svc` shows `<none>` , the Service selector does not match the Pod labels. Compare `app: web` in both files.

## Step 2.4: Watch Kubernetes heal itself

Open a second terminal and run the first command there:

```
# Terminal 2
kubectl get pods -l app=web -w

# Terminal 1
kubectl delete pod -l app=web
```

- `-w` keeps the list updating live. Press Ctrl+C to stop it.
- The second command deletes all the Pods at once.

**You should see:** new Pods appear within seconds. Answer the checkpoint from Part 1: the Deployment's ReplicaSet notices fewer Pods than the file asks for and creates replacements.

### Step 2.5: Change the file, not the cluster

Edit `deployment.yaml` and change `replicas: 3` to `replicas: 5`. Then:

```
kubectl diff -f deployment.yaml
kubectl apply -f deployment.yaml
kubectl get pods -l app=web
```

- `diff` shows what would change (lines with `-` are removed, `+` added) without changing anything.
- `apply` changes only the difference, so running it twice is safe.

**You should see:** 5 Pods. Set the file back to `replicas: 3` and apply again before moving on.

## Part 3: Recreate vs rolling update

**Idea:** when you release a new version, Kubernetes must replace the old Pods. A Deployment can do this in two ways (its *strategy*). **Recreate** stops every old Pod first, then starts the new ones. **RollingUpdate** swaps Pods a few at a time so the app stays online. You will measure the difference.

### Step 3.1: Recreate (see the downtime)

In `deployment.yaml`, set the start of `spec` like this (keep the rest of the file):

```
spec:
  replicas: 4
  strategy:
    type: Recreate
```

```
kubectl apply -f deployment.yaml
```

Open **two more terminals**.

Terminal 2 watches the Pods:

```
kubectl get pods -l app=web -w
```

Terminal 3 plays a user who visits the site twice a second. It runs **inside the cluster**, so it tests the Service itself and works the same on every laptop. Run these three commands there (the first two are needed only once):

```
kubectl run tester --image=curlimages/curl --restart=Never -- sleep 3600
kubectl wait --for=condition=Ready pod/tester
kubectl exec tester -- sh -c "while true; do curl -s -o /dev/null -w '%
{http_code}\n' --max-time 1 web-svc; sleep 0.5; done"
```

**What these commands do**

- `kubectl run tester ...`: starts a small helper Pod that has `curl` installed and stays alive for an hour. (If it says `AlreadyExists`, it is left over from before; just continue.)
- `kubectl wait --for=condition=Ready pod/tester`: pauses until the helper Pod is ready.
- `kubectl exec tester -- sh -c "..."`: runs the loop *inside* the helper Pod. It sends a request to `web-svc` (your Service's name) every half second and prints only the HTTP status. `200` means the site answered. `000` means nobody answered. Press Ctrl+C to stop it.

We test from inside the cluster, not through `localhost`, because it checks the Service itself and behaves the same on every laptop.

```
kubectl set image deployment/web nginx=nginx:1.26
```

`set image deployment/web nginx=nginx:1.26` means: in Deployment `web`, change the container named `nginx` to the image `nginx:1.26`.

**You should see:** in terminal 2, all four old Pods go `Terminating` together, and only then do new Pods start. In terminal 3, a long run of `000` lines appears. **That gap is downtime.**

**Write down:** how many seconds of downtime did you measure? _____

Leave terminals 2 and 3 running.

## Step 3.2: RollingUpdate (no downtime)

Replace the contents of `deployment.yaml` with this complete version. The two new parts are marked:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app: web
```

```
spec:
  replicas: 4
  strategy:                # new
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
          readinessProbe:  # new
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 5
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              cpu: 200m
              memory: 128Mi
```

- `maxSurge: 1`: Kubernetes may run 1 **extra** Pod (5 in total) during the update.
- `maxUnavailable: 0`: Kubernetes may never go **below** 4 ready Pods.
- `readinessProbe`: Kubernetes asks the container "are you ready?" by requesting `/` on port 80 every 5 seconds. A Pod only receives traffic once it answers.

```
kubectl apply -f deployment.yaml
kubectl get pods -l app=web
```

Wait until all 4 Pods show `READY 1/1`. The file says `nginx:1.25`, so this apply also moves the image back to 1.25, and it is already a rolling update.

**Check before you continue:** run `kubectl describe deployment web` and confirm `StrategyType: RollingUpdate`, `RollingUpdateStrategy: 0 max unavailable, 1 max surge`, and a `Readiness: http-get http://:80/` line. If it still says `Recreate`, or the Readiness line is missing, your file was not saved or applied correctly. Fix it and apply again, otherwise the next step will show downtime.

## Step 3.3: Roll out a new version safely

With terminals 2 and 3 still running, in terminal 1:

```
kubectl set image deployment/web nginx=nginx:1.26
kubectl annotate deployment/web kubernetes.io/change-cause="upgrade to nginx
1.26"
kubectl rollout status deployment/web
```

- `annotate ... change-cause`: saves a short reason that appears in the revision history.
- `rollout status`: waits and reports until the update finishes (`successfully rolled out`).

**You should see:** in terminal 2, new Pods start one at a time and old ones stop one at a time, with 4 Pods ready all the time. In terminal 3, **no long run of `000`**. One or two isolated `000` lines can appear when an old Pod stops a moment before the Service stops sending it requests; that is a few requests, not an outage. Compare with your Recreate result.

```
kubectl get rs -l app=web
kubectl rollout history deployment/web
```

**You should see:** two ReplicaSets, the old one at `DESIRED 0` and the new one at `4`. Kubernetes keeps the old one so you can go back instantly.

## Step 3.4: Break it on purpose

```
kubectl set image deployment/web nginx=nginx:9.99
kubectl rollout status deployment/web --timeout=30s
kubectl get pods -l app=web
```

- `nginx:9.99` does not exist, so the new Pod cannot start.
- `--timeout=30s` stops waiting after 30 seconds.

**You should see:** a new Pod with status `ErrImagePull` or `ImagePullBackOff`, **but your 4 old Pods are still `Running` and terminal 3 still prints `200`.** The readiness check and `maxUnavailable: 0` stopped a bad release from taking the site down.

Find the reason: run `kubectl describe pod <the failing pod name>` and read the `Events` at the bottom.

### Step 3.5: Roll back

```
kubectl rollout undo deployment/web
kubectl rollout status deployment/web
kubectl get pods -l app=web
kubectl rollout history deployment/web
```

- `rollout undo` : goes back to the previous working revision.

**You should see:** the failing Pod disappears and 4 healthy Pods run `nginx:1.26`. Press Ctrl+C in terminals 2 and 3 when you finish.

**More rollback commands (optional)**

```
kubectl rollout undo deployment/web --to-revision=1
kubectl rollout pause deployment/web
kubectl rollout resume deployment/web
```

`--to-revision=1` returns to one specific revision; `pause` freezes a rollout part-way and `resume` continues it.

### Compare the two strategies

| | Recreate | RollingUpdate (default) |
|---|---|---|
| Downtime | Yes, while old Pods stop and new ones start | None, if readiness probes are set |
| Versions running at once | One | Two, briefly |
| Use when | Two versions must never run together | Most web apps and APIs |

## Part 4: Practice, cheat sheet and troubleshooting

### Practice tasks

Do these on your own. Use files, not just commands, and keep them in your `k8s-day19` folder.

- [ ] Write a Deployment and a Service for the `httpd` (Apache) image with 2 replicas. Do not copy from the lecture; use `kubectl explain deployment.spec` or your own `deployment.yaml` from Part 2 as a guide if you get stuck.
- [ ] Set `strategy: Recreate` on your `httpd` Deployment. Update the image tag while a curl loop runs and note the seconds of downtime. Repeat with `RollingUpdate` and compare.
- [ ] Change `maxSurge` to 2 and `maxUnavailable` to 1. Update the image and describe how the Pod watch output differs from before.
- [ ] Cause a failed rollout with a wrong tag, recover with `kubectl rollout undo`, and use `kubectl rollout history` to prove which revision is running.
- [ ] In your own words: why does the Service keep working when Pods are replaced and get new IP addresses?

## Command cheat sheet

| Command | What it does |
|---|---|
| `kubectl apply -f file.yaml` | Make the cluster match the file |
| `kubectl apply -f file.yaml --dry-run=client` | Check the file without changing anything |
| `kubectl diff -f file.yaml` | Preview what `apply` would change |
| `kubectl get pods -l app=web -w` | Watch Pods with a label, live |
| `kubectl get deploy,rs,pods` | List Deployments, ReplicaSets and Pods together |
| `kubectl describe pod <name>` | Detailed report; read `Events` at the bottom |
| `kubectl logs <pod>` | Show the application's output |
| `kubectl set image deployment/web nginx=nginx:1.26` | Change the container image (starts a rollout) |
| `kubectl rollout status deployment/web` | Wait for a rollout and report the result |
| `kubectl rollout history deployment/web` | List saved revisions |
| `kubectl rollout undo deployment/web` | Go back to the previous revision |
| `kubectl delete -f file.yaml` | Delete everything defined in the file |

## Troubleshooting

| What you see | Likely cause | What to do |
|---|---|---|
| Pod stays `Pending` | Not enough CPU or memory on your laptop's single node | `kubectl describe pod <name>`; lower `replicas` or `requests` |
| `ErrImagePull` / `ImagePullBackOff` | Wrong image name or tag, or no internet | Check the `image:` line; read `Events` |
| `CrashLoopBackOff` | The container starts and exits | `kubectl logs <pod> --previous` |
| Service gives no answer | Selector does not match the Pod labels | `kubectl get endpoints <service>` shows `<none>`; fix the label |
| `localhost:30080` does not respond | NodePort is not reachable on your setup | `kubectl port-forward svc/web-svc 30080:80` in a second terminal |
| `error converting YAML to JSON` | Tab characters or wrong indentation | Use 2 spaces; check the line number in the error |

## Clean up before you leave

```
kubectl delete pod tester
kubectl delete -f service.yaml -f deployment.yaml
kubectl get all
```

`kubectl get all` should list only the built-in `service/kubernetes`.

**Next lecture:** Helm charts, then blue/green and canary deployments. Keep your YAML files; you will reuse them.